

Introducción:

La información sobre amenazas informáticas se encuentra distribuida entre avisos de seguridad, catálogos de vulnerabilidades y análisis técnicos publicados por distintas organizaciones en diferentes sitios web. Esta dispersión dificulta consultar en un solo lugar qué sistemas pueden verse afectados, cómo se relacionan las amenazas con las vulnerabilidades conocidas y qué medidas se recomiendan para reducir el riesgo. Cosa que puede llegar a dificultar la concientización, formas de defensa personal y efectividad de prevención de ataques cibernéticos.

Por ello, proponemos construir un arañador web y un repositorio de documentos centrados en malware, ransomware y vulnerabilidades relevantes para Windows y Linux. El proyecto permitirá reunir información procedente de diversas fuentes y organizarla para facilitar su consulta y absorción.

Necesidad de Información:

Se necesita reunir y organizar documentos técnicos sobre malware, ransomware y vulnerabilidades relevantes para sistemas de Windows y Linux. La colección debe permitir consultar cómo funcionan las amenazas, qué productos o versiones afectan, cuáles son sus posibles consecuencias y qué medidas de prevención o mitigación existen. También debe incluir fichas de vulnerabilidades CVE, debilidades CWE y patrones de ataque CAPEC.

Además de conservar el texto plano y los datos de origen de cada documento, el repositorio registrará los identificadores CVE, CWE y CAPEC que aparezcan en él. Cuando se detecte un identificador, el arañador podrá incorporar su ficha oficial y vincularla con los documentos que lo mencionan. De esta manera, una consulta podrá recuperar tanto la descripción de un identificador como los avisos y análisis donde se utiliza.

Colectivo al que se dirige

La colección se dirige principalmente a estudiantes de informática y ciberseguridad, personal de soporte técnico y profesionales que administran sistemas Windows o Linux. Como público objetivo inicial se consideran personas de 18 años en adelante.

Existe una alta cantidad de contextos socioeconómicos entre profesionales y estudiantes. Por ello, la colección debe permitir consultar información pública sin depender de suscripciones de pago. Sus subtemas de interés incluyen familias de malware, campañas de ransomware, vulnerabilidades explotadas, sistemas afectados, técnicas de ataque, actualizaciones y recomendaciones defensivas.

Consultas esperadas

La colección debería ayudar a responder preguntas como:

- ¿Qué documentos mencionan este CVE?
- ¿Qué explica la ficha de esta CWE?
- ¿Qué patrón de ataque describe este CAPEC?
- ¿Qué vulnerabilidades se mencionan en el análisis de una familia de ransomware?
- ¿Qué medidas se recomiendan para proteger un sistema afectado?

Políticas de Arañado y Consideraciones de Diseño:

Elementos a usar:

Se usará Python, y una base de datos en PostgreSQL.

Formato e idioma

Solo se usarán documentos HTML en inglés. No se usarán PDF, imágenes, audio, video ni archivos ejecutables. Tampoco se ejecutará JavaScript para obtener contenido que no esté presente en el HTML recibido.

Justificación: Limitar formatos e idioma reduce la complejidad de extracción e indexación.

Metadatos necesarios: Tipo de contenido, idioma detectado y motivo de descarte cuando corresponda (se implementa mediante un if, else, en caso que no cumpla con lo esperado).

Arquitectura sencilla

Se desarrollará una aplicación en Python organizada en módulos: calendarizador, descargadores, extractor, almacenamiento, indexador y buscador. Podrá ejecutarse en un solo equipo junto con PostgreSQL.

La descarga y la indexación se podrán ejecutar como etapas separadas. Con tal de asegurarse de una correcta ejecución se realizará una fase de recopilación de 6 horas, tras de ello, se creará el índice, hasta que este termine se iniciará el siguiente ciclo de recopilación.

Justificación: La separación por responsabilidades facilita implementar y revisar cada parte sin administrar múltiples servicios adicionales. Además de ello, la separación por fases mantiene la implementación estable, y fácil de usar.

Metadatos necesarios: Identificador del ciclo, fecha y hora de inicio y fin de cada fase, estado de ejecución y fecha de finalización del índice.

Fuentes y semillas

Se configurará una lista de diez URLs semilla. Se distinguirá páginas que sirven para descubrir enlaces de documentos que aportan contenido a la colección.

Justificación: Lo dice el documento y especificación del proyecto.

Metadatos necesarios: URL descubierta, sitio, URL de procedencia, fecha de descubrimiento y condición de semilla.

Configuración: Lista de semillas.

Acceso público y conexión HTTPS

Se descargarán páginas públicas y gratuitas mediante HTTPS con validación del certificado. Se excluirán contenidos que requieran autenticación, pago o superar controles de acceso.

Justificación: La colección debe atender a personas que dependen de recursos públicos y permitir verificar la procedencia de los documentos.

Metadatos necesarios: URL, resultado de acceso y motivo de rechazo.

Respeto a robots.txt y restricciones de acceso

Antes de solicitar páginas de un sitio, el arañador consultará su archivo robots.txt. Identificará las reglas aplicables a su agente de usuario y comprobará qué rutas puede recorrer mediante las directivas Allow y Disallow. Las rutas prohibidas no se incorporarán al recorrido.

Se utilizará un intervalo predeterminado de tres segundos entre inicios de solicitudes al mismo host. Cuando exista un Crawl-delay válido y aplicable, ese valor sustituirá el intervalo predeterminado, tanto si es mayor como si es menor. En todos los casos se mantendrá una sola solicitud activa por host.

Además, se mantendrá el máximo de una solicitud activa por sitio. Por tanto, una nueva solicitud sólo podrá comenzar cuando haya terminado la anterior y se haya cumplido la espera correspondiente. Las pausas por errores o por Retry-After también deberán cumplirse antes de continuar.

Las reglas de robots.txt se conservarán durante un máximo de 24 horas desde su última consulta satisfactoria, de forma independiente para cada sitio. Una vez vencido ese período, se volverá a consultar el archivo antes de solicitar otra página del sitio. Si no hay solicitudes pendientes hacia ese sitio, no será necesario revisarlo hasta que se retome su recorrido.

Si robots.txt devuelve 404, se continuará aplicando las demás políticas del proyecto. Ante un fallo temporal de conexión o del servidor, se aplazará el recorrido del sitio y se aplicará la política de reintentos. Si se agotan los intentos sin poder comprobar las reglas, el sitio quedará pendiente de revisión posterior.

Justificación: Esta política busca respetar las restricciones publicadas por las fuentes y evitar una frecuencia de acceso superior a la permitida. El intervalo de tres segundos proporciona un límite inicial cuando el sitio no publica uno.

Metadatos necesarios: Sitio, reglas obtenidas, fecha de consulta, resultado HTTP, espera indicada cuando exista y próxima fecha de revisión.

Configuración: Agente de usuario, intervalo base de tres segundos y periodo de revisión de robots.txt.

Descubrimiento y normalización de enlaces

Se extraerán enlaces del HTML y se resolverán las direcciones relativas. Se eliminarán los fragmentos de navegación, como #mitigaciones, para evitar descargar varias veces una misma página.

Solo se eliminarán parámetros conocidos de seguimiento. Se conservarán los parámetros que puedan identificar contenidos diferentes.

Se excluirán enlaces a formularios, autenticación, búsquedas internas y combinaciones repetitivas de filtros.

Justificación: Estas reglas reducen duplicados y recorridos interminables sin confundir documentos distintos.

Metadatos necesarios: URL original, URL normalizada, procedencia y motivo de exclusión.

Un arañador con cuatro descargadores

Una ejecución tendrá cuatro descargadores concurrentes, implementados mediante hilos en el arañador propio. Compartirán la cola, las reglas y el control de solicitudes.

Los artículos y las fichas CVE, CWE y CAPEC serán descargados por estos mismos trabajadores.

Justificación: Permite avanzar mientras otras solicitudes esperan respuesta y evita mantener un arañador adicional para las fichas.

Metadatos necesarios: URL asignada, estado y ejecución responsable de la tarea.

Configuración: Número de descargadores: 4

Frecuencia y concurrencia por sitio

Se permitirá como máximo una solicitud activa por host y cuatro solicitudes simultáneas en todo el arañador, correspondientes a sus cuatro descargadores.

Se utilizará un intervalo predeterminado de tres segundos entre inicios de solicitudes al mismo host. Cuando exista un Crawl-delay válido y aplicable en robots.txt, su valor sustituirá ese intervalo, tanto si es mayor como si es menor. Los subdominios de una fuente podrán agruparse bajo un mismo límite si corresponde.

El calendarizador aplicará estas restricciones de forma compartida a todos los descargadores. Una nueva solicitud al mismo host solo podrá comenzar cuando haya terminado la anterior, haya transcurrido el intervalo efectivo y se hayan cumplido las pausas vigentes, incluidas las indicadas mediante Retry-After.

Justificación: Controlar la simultaneidad y la frecuencia limita la carga sobre las fuentes. Los cuatro descargadores permiten trabajar con diferentes sitios durante las esperas. El intervalo predeterminado de tres segundos es una decisión inicial del proyecto, no un valor universal establecido por un estándar.

Metadatos necesarios: Host, inicio de la última solicitud, espera indicada por el sitio cuando exista y próxima solicitud permitida, considerando las pausas vigentes. Los contadores de solicitudes activas podrán mantenerse en memoria durante la ejecución.

Configuración: Intervalo predeterminado de tres segundos, máximo de una solicitud simultánea por host y máximo de cuatro solicitudes simultáneas en todo el arañador.

Política de redirecciones

El arañador seguirá un máximo de cinco redirecciones consecutivas para obtener una página. Cada cambio de dirección constituye un salto; $A \rightarrow B \rightarrow C$ representa dos redirecciones. Antes de seguir cada destino se comprobará que cumple las políticas de acceso.

Si se detecta un ciclo, como $A \rightarrow B \rightarrow A$, se interrumpirá el seguimiento inmediatamente. También se interrumpirá cuando se requiera una sexta redirección. Se registrará el motivo y se continuará con otra URL pendiente.

Justificación: Permite seguir cambios de ubicación de documentos y evita solicitudes repetitivas causadas por ciclos o cadenas excesivas.

Metadatos necesarios: URL inicial, URL final alcanzada, cantidad de redirecciones y motivo de interrupción. Las direcciones recorridas durante el intento podrán mantenerse temporalmente en memoria para detectar ciclos.

Política de tamaño máximo de descarga

Se aceptarán respuestas HTML de hasta 10 MB —10 000 000 bytes—, medidos sobre el contenido descomprimido. Cuando se supere este límite, se interrumpirá la descarga, se

eliminará el contenido parcial y se registrará la URL como descartada por tamaño. El descargador continuará con otra dirección pendiente.

Justificación: Un HTML grande puede tomar mucha memoria y atención de un descargador. Así que se define un límite de 10MB, así no se gasta tanta banda de memoria y tiempo en una sola página que podría o no contener información útil.

Metadatos necesarios: URL, cantidad de bytes recibidos hasta la interrupción, estado y motivo de descarte.

Política de intentos y tiempos de espera

Cada URL dispondrá de un máximo de tres intentos de descarga, con un plazo máximo de 30 segundos por intento:

Intento	Espera previa
Inicial	La necesaria para cumplir las restricciones del sitio.
Segundo	Sin una pausa adicional por reintento, pero respetando el intervalo mínimo entre solicitudes.
Tercero y último	Al menos 60 segundos después del fallo del segundo intento.

Todos los intentos respetarán el intervalo efectivo del sitio: el indicado por un Crawl-delay válido o, en su ausencia, tres segundos. El segundo intento no tendrá una pausa adicional. El tercero comenzará al menos 60 segundos después del segundo fallo. Si existe una pausa indicada mediante Retry-After, también se respetará. La solicitud solo se iniciará cuando se hayan cumplido todas las esperas aplicables.

Los reintentos se realizan únicamente ante fallos temporales. Al agotar los tres intentos, la URL quedará registrada como fallida y se continuará con otras direcciones.

Justificación: Permite recuperarse rápidamente de un fallo ocasional, incorpora una pausa antes del último intento y evita insistir indefinidamente.

Metadatos necesarios: URL, cantidad de intentos realizados, último código HTTP o error, próxima fecha permitida de intento y estado final.

Cola persistente y calendarizador a corto plazo

La lista de URLs pendientes se guardará en cola.txt, con una dirección normalizada por línea. PostgreSQL conservará sus estados, intentos y fechas de reintento o revisita. Únicamente el calendarizador modificará el archivo y asignará las URLs a los descargadores.

Un calendarizador central seleccionará la URL pendiente más antigua que cumpla las restricciones en ese momento. Si su sitio está en pausa, podrá seleccionar otra dirección disponible.

La reserva de la URL se registrará antes de asignarla a un descargador. No se mantendrá una transacción de base de datos abierta durante la descarga.

Justificación: La cola persistente permite reanudar el trabajo; la asignación central evita reservas duplicadas y coordina los cuatro descargadores.

Metadatos necesarios: URL normalizada, host, estado, fecha de incorporación, próxima fecha de intento y ejecución que la reservó.

Calendarizador a largo plazo

Los avisos y análisis (es decir todo lo recopilado hasta el momento que no sean códigos CWE, CVE o CAPEC) se revisarán inicialmente cada mes y las fichas cada 2 meses. Al iniciar una ejecución, el calendarizador incorporará las revisitas vencidas.

Las revisiones se programarán a partir de la última comprobación satisfactoria. Los fallos se manejan mediante la política de reintentos.

Los listados utilizados para descubrir publicaciones se volverán a consultar al inicio de cada ciclo de recopilación. Sus enlaces nuevos se incorporarán a la cola, mientras que las URLs ya registradas conservarán sus estados y fechas programadas. Los listados no se incorporarán al índice salvo que también cumplan los criterios de aceptación como documentos.

Justificación: Permite actualizar documentos sin mantener el programa funcionando permanentemente ni consultar cada página de manera continua.

Metadatos necesarios: Tipo de documento, última comprobación satisfactoria y próxima revisión.

Configuración: Periodos de revisita.

Extracción del texto plano

Se utilizará una biblioteca de análisis y extracción HTML. Se conservarán el contenido principal, títulos, párrafos, listas y texto de tablas cuando puedan extraerse correctamente.

Se eliminarán menús, anuncios y contenido repetitivo ajeno al documento. Se normalizarán espacios y saltos de línea, almacenando el texto en UTF-8.

Justificación: La colección debe conservar explicaciones útiles para el colectivo, incluidos flujos de ataque y mitigaciones, sin que la navegación del sitio domine las búsquedas.

Metadatos necesarios: Resultado de extracción, título, ruta del texto y tamaño en bytes.

Comprobación de idioma y pertinencia

El idioma se detectará sobre el texto principal. Se descartaran documentos vacíos, de otros idiomas o con extracción insuficiente. Esto con tal de evitar información sin importancia, documentos vacíos, entre otros.

Justificación: Este procedimiento permite filtrar la colección sin entrenar un clasificador específico.

Metadatos necesarios: Idioma detectado, aceptación o descarte y motivo.

Configuración: Reglas temáticas y criterio de aceptación del detector de idioma.

Detección de identificadores

Se reconocerán mediante patrones los identificadores CVE, CWE y CAPEC presentes en el texto plano, incluido el texto visible de enlaces. Se normalizan (CASE INSENSITIVE) para registrarlos de forma consistente.

Cada identificador se registrará una sola vez por versión, aunque aparezca repetidamente.

Justificación: Permite recuperar documentos por códigos concretos y descubrir fichas sin depender de que el autor haya incluido un enlace.

Metadatos necesarios: Código normalizado, tipo de identificador y versión del documento donde aparece.

Incorporación de fichas

Para cada identificador se construirá su dirección mediante una plantilla configurable de la fuente correspondiente. La ficha se añadirá a la cola si todavía no está registrada para descarga o revisita.

Se aplicarán las mismas reglas de acceso y dosificación que al resto de las páginas. Si la ficha no se obtiene, se conservará la mención y se registrará el fallo.

Las menciones encontradas en fichas aceptadas también podrán incorporar otras fichas mediante el mismo procedimiento y control de duplicados.

Justificación: Facilita ampliar la información disponible sobre códigos encontrados sin descargar repetidamente una ficha.

Metadatos necesarios: Identificador, URL de ficha, documento de ficha cuando exista y estado de obtención.

Configuración: Plantillas de URL. No se asumirá que las direcciones permanecerán invariables.

Relaciones mediante menciones

La relación almacenada será exclusivamente «esta versión menciona este identificador». Los documentos se podrán relacionar por compartir códigos.

No se infiere una relación técnica entre un CVE, una CWE y un CAPEC por aparecer juntos.

Justificación: Resuelve las consultas previstas con una relación sencilla y verificable en el texto.

Metadatos necesarios: Identificador de versión y código mencionado, con una restricción que impida repetir la misma pareja.

Control de URLs ya visitadas

El arañador conservará en PostgreSQL las URLs normalizadas y su estado. Antes de agregar una dirección a la cola, comprobará si ya está registrada. Si está pendiente, en proceso o completada, no la agrega nuevamente. Las revisitas y los reintentos se realizarán únicamente cuando correspondan según sus respectivas políticas.

No se comprobará si documentos provenientes de URLs diferentes tienen el mismo contenido.

Justificación: Evita procesar repetidamente una misma dirección y mantiene sencilla la implementación. Se acepta que puedan almacenarse textos iguales publicados en distintas URLs.

Metadatos necesarios: URL normalizada, estado y próxima fecha de visita o reintento cuando corresponda.

Réplica local e historial

Cada documento aceptado conservará su HTML y texto plano localmente. No se descargan automáticamente imágenes ni otros recursos necesarios para reproducir visualmente toda la página. Se utilizará sobre el texto plano una huella SHA-256, para el manejo de versiones.

Se conservará la versión vigente y hasta dos anteriores. Una nueva versión se creará cuando cambie la huella del texto plano. Si no cambia, únicamente se actualizará la fecha de comprobación.

Al superar las tres versiones, se retirará la más antigua y sus registros dependientes.

Justificación: La copia local permite consultar y procesar el documento; el historial permite revisar cambios con un límite de almacenamiento.

Metadatos necesarios: Documento, versión, fecha, huella, rutas del HTML y texto, y versión vigente.

“Texto enriquecido”

El texto plano se acompañará de URL original, fechas disponibles y menciones de códigos. Estos elementos se almacenarán como campos separados. No es texto enriquecido como tal.

La fecha de descarga no se presentará como si fuera la fecha de publicación. Si un dato no aparece en la fuente, se dejará sin valor.

Justificación: Aporta contexto y procedencia sin introducir información inventada.

Metadatos necesarios: URL original, fechas de publicación y descarga cuando correspondan, y menciones de código.

Uso de PostgreSQL

PostgreSQL almacenará los metadatos, los estados de las URLs y las relaciones entre documentos, versiones e identificadores. La cola de URLs pendientes estará en cola.txt. Los archivos HTML y de texto permanecerán en carpetas separadas y la base de datos conservará sus rutas.

Una URL se registrará en documentos desde su descubrimiento, aunque todavía no haya sido descargada. Tendrá versiones únicamente cuando se haya obtenido y aceptado contenido.

Se utilizarán las siguientes tablas:

Tabla sitios

Una fila representará un host consultado.

Columna	Función
id	Identificador del sitio.

host	Nombre del host, sin repetirlo.
permitido	Indica si está autorizado para el recorrido.
robots_txt	Contenido de las reglas obtenidas.
fecha_revision_robots	Momento de la última consulta de esas reglas.
proxima_revision_robots	Momento en que corresponde actualizarlas.
codigo_http_robots	Resultado de la consulta de robots.txt.
espera_indicada_segundos	Espera publicada por el sitio, si existe.
ultima_solicitud	Inicio de la última solicitud al host.
proxima_solicitud_permitida	Momento a partir del cual podrá iniciarse otra solicitud, considerando intervalos y pausas.

El número de solicitudes activas se mantendrá en memoria durante la ejecución.

Tabla documentos

Una fila representará una URL administrada por el arañador.

Columna	Función
id	Identificador del documento o URL registrada.

url_original	Dirección tal como fue descubierta por primera vez.
url_normalizada	Dirección utilizada para evitar incorporar repetidamente la misma URL.
url_final	Última dirección alcanzada después de redirecciones.
url_procedencia	Página desde la que se descubrió la URL.
fecha_descubrimiento	Momento de incorporación al sistema.
es_semilla	Indica si pertenece a las URLs iniciales.
tipo	Listado de descubrimiento, aviso/análisis o ficha.
estado	Pendiente, en proceso, completada, descartada o fallida.
intentos	Intentos realizados para la tarea actual; se reinician al comenzar una revisita nueva.
proximo_intento	Fecha permitida para un reintento.
ultima_revision	Última comprobación satisfactoria.
proxima_revision	Fecha programada de revisita.
ultimo_codigo_http	Resultado HTTP del último intento.

ultimo_error	Descripción breve del último fallo, si existe.
motivo_descarte	Razón del rechazo, cuando corresponda.
tipo_contenido	Tipo de contenido recibido.
idioma_detectado	Idioma detectado, incluso cuando provoque el descarte.
resultado_extraccion	Indica si se obtuvo texto correctamente, quedó vacío o falló la extracción.
cantidad_redirecciones	Saltos seguidos en el último intento.
bytes_ultimo_intento	HTML descomprimido recibido, incluso si la descarga se interrumpió.
idCiclo	Ciclo que asignó la tarea más reciente.

No se volverá a agregar una URL descartada o fallida simplemente porque aparezca otro enlace hacia ella. Los reintentos o visitas deberán programarse expresamente.

Tabla versiones

Una fila representará una versión aceptada de un documento.

Columna	Función
id	Identificador de la versión.
idDoc	Referencia al documento.

fecha_descarga	Momento en que se obtuvo esa versión.
titulo	Título agregado por fines estéticos.
fecha_publicacion	Fecha indicada por la fuente, si está disponible.
idioma	Idioma de esa versión.
ruta_html	Ubicación del HTML guardado.
ruta_texto	Ubicación del texto plano.
hash_texto	Huella SHA-256 para comprobar cambios de la misma página.
tamano_html_bytes	Tamaño del HTML almacenado.
tamano_texto_bytes	Tamaño del texto plano almacenado.

Se conservará una sola versión vigente y hasta dos anteriores por documento.

Cada documento tendrá como máximo una versión marcada como vigente. La nueva versión se marcará como vigente después de completar correctamente su almacenamiento.

Tabla identificadores

Columna	Función
codigo	Código normalizado y único, por ejemplo, CWE-78.

tipo	CVE, CWE o CAPEC.
url_ficha	Dirección construida para obtener la ficha.
idDocumentoFicha	Referencia al registro de la ficha en documentos.

Tabla menciones

Columna	Función
idVersion	Versión donde aparece la mención.
codigoidentificador	Código mencionado.

La combinación de idVersion y codigoidentificador será única, para evitar registrar repetidamente el mismo código en una versión.

Tabla ciclos

Se incorpora esta tabla pequeña porque el apartado de arquitectura exige registrar el inicio, fin y estado de las fases.

Columna	Función
id	Identificador del ciclo.
implementacion	Arañador propio o implementación con bibliotecas.

inicio_recopilacion	Inicio de las seis horas de recopilación.
fin_recopilacion	Finalización real de esa fase.
inicio_indexacion	Inicio de construcción del índice.
fin_indexacion	Finalización de la indexación.
estado	Recopilando, indexando, completado, interrumpido o fallido.

La relación entre las filas del índice y las versiones indexadas permanecerá en los archivos del índice. El historial de solicitudes y sus resultados permanecerá en el log; las columnas de documentos conservarán el estado más reciente.

Justificación: Esta estructura permite aplicar las políticas de acceso, reintentos, revisitas, versiones y menciones. La tabla de ciclos permite identificar tareas interrumpidas y registrar las dos fases sin repetir sus fechas en cada documento.

Metadatos necesarios: Los campos descritos en las tablas. Las fechas desconocidas y los campos que todavía no correspondan permanecerán en NULL. Se aplicarán claves foráneas y restricciones de unicidad para mantener las relaciones.

Consideraciones de Uso de las Tablas:

- **Nombres:** idDoc, idCiclo e idDocumentoFicha se escribieron como id_doc, id_ciclo e id_documento_ficha para mantener una nomenclatura uniforme.
- **Fechas:** TIMESTAMPTZ almacena fechas y horas con tratamiento de zona horaria. fecha_publicacion puede quedar vacía si la fuente no la proporciona.
- **Versiones:** el programa conservará las tres más recientes y eliminará las anteriores junto con sus archivos. Eliminar una fila de PostgreSQL no elimina los archivos del disco.
- **Menciones:** la clave primaria compuesta impide registrar varias veces el mismo identificador en una misma versión. El código identificador es un texto.
- **Índice vectorial:** los archivos TF-IDF se almacenarán por separado; no se agrega una columna embedding.

Integridad y acceso a PostgreSQL

PostgreSQL almacenará los metadatos de sitios, documentos y versiones, así como los identificadores y sus menciones. Se utilizarán claves primarias, claves foráneas y

restricciones de unicidad para mantener las relaciones y evitar registros duplicados. Las consultas desde Python serán parametrizadas y las operaciones relacionadas se agruparán en transacciones breves.

La lista de URLs pendientes se almacenará en un archivo TXT, con una dirección normalizada por línea. Únicamente el calendarizador modificará este archivo y asignará las URLs a los descargadores. PostgreSQL conservará el estado de cada URL, sus intentos y las fechas necesarias para controlar reintentos y revisitas.

Antes de asignar una URL, el calendarizador comprobará sus restricciones y la marcará como en_proceso en PostgreSQL. Después actualizará el TXT. Al reiniciar, se comprobará la coherencia entre ambos almacenamientos: se recuperarán las tareas interrumpidas y se excluirán las URLs completadas cuya revisita aún no corresponda.

Justificación: El TXT permite mantener una cola sencilla y legible, mientras PostgreSQL conserva los metadatos y relaciones. Centralizar las modificaciones del archivo evita que los cuatro descargadores tomen la misma dirección o sobrescriban cambios.

Metadatos necesarios: URL normalizada, estado, número de intentos, próxima fecha de intento o revisita e identificador de la ejecución que recibió la tarea.

Separación de almacenamientos

Almacenamiento	Contenido
PostgreSQL	Metadatos, estados de URLs, documentos, versiones, identificadores y menciones.
Archivo cola.txt	URLs pendientes de descarga, una por línea.
Carpeta de HTML	Copias de las páginas aceptadas.
Carpeta de texto	Texto plano de las versiones conservadas.
Carpeta del índice	Vocabulario, pesos TF-IDF y referencias a documentos y versiones.

Carpeta de registros	Bitácoras y estadísticas de ejecución.
Archivo de configuración	Semillas, fuentes permitidas, límites y parámetros generales.

Los archivos de documentos se nombran mediante sus identificadores internos y versiones. PostgreSQL conservará sus rutas.

El calendarizador actualizará la cola escribiendo primero un archivo temporal y sustituyendo después el TXT anterior, para evitar dejarlo parcialmente escrito. Las URLs que queden pendientes al finalizar un ciclo se conservarán para el siguiente.

Justificación: La separación facilita localizar y administrar cada tipo de información. Todos los almacenamientos podrán estar en el mismo equipo, manteniendo sencilla la instalación.

Metadatos necesarios: Rutas de los archivos de cada versión y correspondencia entre entradas del índice, documentos y versiones. La ubicación de cola.txt se establecerá en la configuración.

Modelo vectorial clásico

Se propone representar los documentos mediante TF-IDF (Term Frequency x Inverse Document Frequency) y ordenarlos respecto de una consulta mediante similitud del coseno.

Se usará el documento completo en su versión vigente. Se conservará una representación dispersa, almacenando principalmente los pesos distintos de cero.

Justificación: Permite implementar un modelo vectorial comprensible sin incorporar modelos de embeddings ni fragmentación por límites de entrada. La representación dispersa evita almacenar grandes cantidades de ceros.

Metadatos necesarios: Correspondencia entre cada fila del índice y el documento y versión representados.

Datos del índice: Vocabulario, pesos IDF y vectores TF-IDF. Estos son parte del índice, se deben guardar en este.

Tokenización y normalización

El indexador separará el texto en términos y será case insensitive. Reconocerá primero los identificadores CVE, CWE y CAPEC para conservarlos como unidades completas.

No se eliminarán indiscriminadamente números o signos internos que formen parte de versiones de productos o identificadores técnicos. Las consultas recibirán el mismo tratamiento que los documentos.

Justificación: La normalización permite comparar términos equivalentes sin destruir información técnica importante para el colectivo.

Metadatos necesarios: No requiere metadatos específicos por documento.

Configuración: Reglas de tokenización y normalización.

Stopwords

Se eliminará una lista fija y configurable en un txt de stopwords en inglés, como artículos y otras palabras de escaso valor discriminativo.

Se conservarán negaciones como *no*, *not* y *nor*. Los identificadores técnicos no se eliminarán mediante esta lista.

La eliminación se aplicará únicamente a la representación destinada al índice y a las consultas.

Justificación: Reduce términos poco útiles para diferenciar documentos. Conservar negaciones evita eliminar información relevante, aunque el modelo clásico no interprete por completo el significado de las oraciones.

Metadatos necesarios: No se necesitan metadatos particulares por documento.

Configuración: Lista de stopwords y excepciones.

Lematización y stemming

No se aplicará lematización ni stemming. Palabras como *attack* y *attacks* permanecerán como términos distintos.

Justificación: Reduce herramientas y reglas adicionales y evita transformaciones incorrectas de términos técnicos. Y simplifica el proyecto.

Metadatos necesarios: Ninguno específico, porque estos procesos no se implementarán.

Conservación del texto legible

El texto plano conservado en archivos no perderá stopwords ni se convertirá permanentemente a minúsculas. El indexador generará su representación procesada a partir de ese texto.

Justificación. Permite mostrar documentos legibles y reconstruir el índice sin descargar nuevamente las páginas.

Metadatos necesarios. Ruta del texto plano y referencia a su versión.

Construcción y actualización del índice

El índice se construirá por lotes sobre la versión vigente de cada documento. No se comparará el contenido de URLs diferentes para eliminar duplicados; por tanto, documentos con el mismo texto y distintas URLs podrán quedar indexados.

Recordemos, que la recolección se realizará durante 6 horas y luego se detendrá, para dar lugar a la creación del índice. Donde, el nuevo índice sustituirá al anterior únicamente cuando termine correctamente.

Justificación: La reconstrucción simplifica la consistencia entre vocabulario, pesos y documentos. Mantener el índice anterior hasta completar el nuevo evita dejar la búsqueda con una construcción incompleta.

Metadatos necesarios: Fecha de construcción y relación de documentos y versiones incluidos.

Análisis de consultas

Se admitirán consultas temáticas en inglés y consultas por identificadores.

Una consulta formada únicamente por un código se resolverá mediante coincidencia exacta en PostgreSQL. En consultas que combinan códigos y palabras, los códigos se utilizarán como filtros y las palabras restantes para ordenar por similitud.

Las consultas temáticas se procesarán con las mismas reglas del índice y utilizarán su vocabulario y pesos IDF.

Justificación: Las menciones permiten resultados exactos para códigos; TF-IDF permite ordenar documentos por términos temáticos sin depender de los identificadores.

Metadatos necesarios: Se utilizarán las menciones y referencias existentes. No será necesario guardar permanentemente cada consulta.

Conjunto de respuesta y ordenamiento

Las consultas temáticas devolverán inicialmente hasta 50 documentos únicos con similitud positiva, ordenados por puntuación. Los empates se resolverán mediante el identificador interno para mantener un orden estable.

En búsquedas exactas se presentará primero la ficha, si está disponible, y después los documentos que mencionan el código.

Si no quedan términos útiles o ninguno pertenece al vocabulario, el sistema solicitará reformular la consulta. No presentará documentos con puntuación cero como si fueran relevantes.

Justificación: El conjunto limitado facilita una primera implementación y evita resultados repetidos o sin coincidencia. La puntuación representa semejanza textual, no peligrosidad ni veracidad.

Metadatos necesarios: Identificador de documento, título, fechas y referencia a la versión. Las puntuaciones se calcularán para cada consulta y no requieren almacenamiento permanente.

Extracción y dosificación de resultados

Se mostrarán diez documentos por página. Cada resultado incluirá fuente, URL, fecha disponible e identificadores mencionados.

Justificación: Permite al usuario evaluar la utilidad de cada resultado y consultar su procedencia.

Metadatos necesarios: Los campos descriptivos existentes y la ruta del texto.

Consulta de versiones históricas

La búsqueda normal utilizará versiones vigentes. Desde un documento se podrán consultar las versiones anteriores conservadas, con sus fechas y menciones.

Justificación: Evita mezclar información histórica con resultados actuales y permite revisar cambios cuando el usuario lo necesite.

Metadatos necesarios: Documento, versiones conservadas, fechas y versión vigente.

Bitácora y estadísticas básicas

Se guardará un log por ciclo. La fase de recopilación durará hasta seis horas; la duración total del ciclo incluirá también la indexación. Cada solicitud registrará únicamente fecha y hora, URL y resultado, para demostrar el recorrido del arañador. Al finalizar se incluirá un resumen con:

- Cantidad de URLs distintas consultadas.
- Cantidad de documentos descargados completamente.
- Cantidad de documentos aceptados y descartados.
- Tamaño total del HTML recibido y del texto plano almacenado, por separado.
- Cantidad de errores y reintentos.
- Duración de la ejecución.

Al finalizar la recopilación utilizada para la entrega, se calcularán por separado las estadísticas del repositorio: tamaño total del texto plano vigente, cantidad de documentos, cantidad total de palabras, cantidad de palabras distintas y gráfica de sus frecuencias. Para estos conteos se normalizarán mayúsculas y minúsculas, pero se conservarán las stopwords y no se aplicará lematización.

Justificación: Permite comprobar el recorrido exigido por el proyecto y comparar las dos implementaciones sin generar registros excesivamente detallados.

Metadatos necesarios: Fecha y hora, URL, resultado, tamaño recibido y tamaño del texto plano almacenado. Los contadores se acumularán durante la ejecución.

Interrupción y reanudación

Al detenerse el sistema se conservarán la cola y el trabajo terminado. Al reiniciar, las tareas de una ejecución interrumpida que hayan quedado en proceso volverán a estar pendientes.

Los archivos se escribirán temporalmente y se marcarán como disponibles después de completar su escritura. Un documento no se publicará como versión vigente hasta que sus archivos y registros estén preparados.

Justificación: Evita utilizar archivos incompletos y permite continuar sin repetir toda la recopilación.

Metadatos necesarios: Estado, ejecución responsable y referencias a archivos completos.

Volumen de la colección y condiciones de parada

El volumen se calculará sumando los bytes del texto plano de una sola versión vigente por URL normalizada, sin compresión. No se incluirán HTML, versiones anteriores, índices ni respaldos. Como no se eliminarán duplicados por contenido entre URLs diferentes, esta medida no garantiza que todos los textos contabilizados sean distintos.

Cada fase de recopilación finalizará al cumplir seis horas, agotar las tareas pendientes, recibir una orden de detención o encontrar un fallo de almacenamiento que impida conservar correctamente los resultados.

Justificación: Permite medir el texto disponible en la colección sin contar versiones históricas ni archivos auxiliares, manteniendo el control de duplicados únicamente por URL.

Metadatos necesarios: Tamaño del texto, huella, versión vigente y estado de la cola.

Validación y comparación de implementaciones

Para comparar el arañador propio con una implementación mediante un framework, se utilizarán las mismas semillas, reglas y condiciones de parada. Se ejecutarán por separado y se identificarán sus resultados para evitar que una ejecución condicione la cola de la otra.

Se comprobarán bajo los mismos lineamientos de este documento.

Cada implementación utilizará su propia base de datos con la misma estructura, su archivo cola.txt y sus carpetas de salida. Ambas comenzarán las pruebas comparativas desde condiciones equivalentes, para que los estados y documentos obtenidos por una no alteren el recorrido de la otra.

Justificación: Permite evaluar el cumplimiento de las políticas y la utilidad de la colección para las consultas previstas, además de comparar implementaciones bajo condiciones equivalentes.

Metadatos necesarios: Identificador de ejecución, implementación utilizada, condiciones aplicadas y estadísticas obtenidas.

Archivo de configuración

Se utilizará un archivo JSON que reúna los parámetros generales de ambas implementaciones. Los valores que cambian durante la ejecución, como estados, intentos realizados y fechas de revisión, se almacenarán en PostgreSQL o en la bitácora.

La configuración incluirá (esto solo se hace por resumen):

Grupo	Parámetro	Valor o contenido
Fuentes	Semillas	Lista de al menos diez URLs iniciales.
Fuentes	Sitios y rutas permitidos	Fuentes y secciones autorizadas para el recorrido.
Fuentes	Rutas excluidas	Autenticación, búsquedas internas y otras rutas descartadas.
Acceso	Agente de usuario	Nombre que identificará al arañador.
Acceso	Protocolo	HTTPS con validación del certificado.
Acceso	Revisión de robots.txt	86 400 segundos —24 horas— por sitio.
Descarga	Descargadores concurrentes	4.

Descarga	Solicitudes activas por host	1.
Descarga	Espera predeterminada	3 segundos entre inicios de solicitudes al mismo host.
Descarga	Espera indicada por el sitio	Utilizar el Crawl-delay válido cuando exista, según la política del documento.
Descarga	Tiempo máximo por intento	30 segundos.
Descarga	Intentos totales por URL	3: inicial y dos reintentos.
Descarga	Espera adicional antes del segundo intento	0 segundos; se mantienen las restricciones de acceso vigentes.
Descarga	Espera adicional antes del tercer intento	60 segundos desde el segundo fallo.
Descarga	Pausas del servidor	Respetar Retry-After.
Descarga	Redirecciones máximas	5; interrumpir antes si se detecta un ciclo.
Descarga	Tamaño máximo del HTML	10 000 000 bytes descomprimidos.
Ciclos	Duración de recopilación	21 600 segundos: seis horas.

Ciclos	Inicio de indexación	Después de cerrar la fase de recopilación y terminar las tareas activas.
Ciclos	Duración de indexación	Hasta completar el índice; sin duración fija.
Revisitas	Avisos y análisis	Un mes desde la última comprobación satisfactoria.
Revisitas	Fichas	Dos meses desde la última comprobación satisfactoria.
Revisitas	Listados de descubrimiento	Al inicio de cada ciclo de recopilación.
Versiones	Máximo conservado	3: vigente y dos anteriores.
Extracción	Formato admitido	HTML.
Extracción	Idioma admitido	Inglés.
Extracción	Codificación del texto almacenado	UTF-8.
Extracción	Filtro temático	Reglas o términos utilizados para comprobar pertinencia.
Identificadores	Patrones de detección	CVE, CWE y CAPEC, sin distinguir mayúsculas.

Identificadores	Plantillas de fichas	Regla de construcción de URL para cada fuente.
URLs	Parámetros de seguimiento eliminables	Lista explícita de parámetros que pueden retirarse.
Indexación	Modelo	TF-IDF con similitud del coseno.
Indexación	Unidad indexada	Documento completo en su versión vigente.
Indexación	Normalización	Sin distinción de mayúsculas, conservando códigos técnicos completos.
Indexación	Stopwords	Ruta del archivo TXT que contiene la lista.
Indexación	Excepciones de stopwords	no, not, nor y términos técnicos protegidos.
Indexación	Lematización y stemming	Desactivados.
Consultas	Máximo de resultados temáticos	50 documentos.
Consultas	Resultados por página	10.
Almacenamiento	Cola	Ruta de cola.txt.

Almacenamiento	Documentos	Rutas de las carpetas de HTML y texto.
Almacenamiento	Índice y registros	Rutas de las carpetas correspondientes.
PostgreSQL	Conexión	Host, puerto y nombre de la base de datos; credenciales mediante variables de entorno.

Los períodos expresados en meses se calcularán como **meses calendario**, para evitar que distintas partes del programa los interpreten de manera diferente.

No se establecerá un intervalo periódico para actualizar cola.txt: se actualizará cuando se incorporen o retiren URLs. Las URLs nuevas obtenidas de una misma página se agregarán conjuntamente.

Justificación: Centralizar estos parámetros permite modificar el comportamiento sin alterar múltiples partes del código y utilizar las mismas políticas en las dos implementaciones.

Metadatos necesarios. Este apartado no requiere metadatos por documento. Los parámetros enumerados son configuración; los resultados de su aplicación se registran en las tablas y en la bitácora.

Urls Semillas:

<https://www.cisa.gov/news-events/cybersecurity-advisories>
<https://www.microsoft.com/en-us/security/blog/topic/threat-intelligence/>
<https://ubuntu.com/security/notices>
<https://www.debian.org/security/>
<https://blog.talosintelligence.com/>
<https://unit42.paloaltonetworks.com/>
<https://securelist.com/>
<https://nvd.nist.gov/vuln/detail/CVE-2017-0144>
<https://cwe.mitre.org/data/definitions/79.html>
<https://capec.mitre.org/data/definitions/100.html>

Anexos:

Creación de Tablas PostgreSQL:

```
BEGIN;
```

```
-- 1. Hosts autorizados y reglas de acceso.
```

```

CREATE TABLE sitios (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    host TEXT NOT NULL UNIQUE,
    permitido BOOLEAN NOT NULL DEFAULT FALSE,

    robots_txt TEXT,
    fecha_revision_robots TIMESTAMPTZ,
    proxima_revision_robots TIMESTAMPTZ,
    codigo_http_robots INTEGER,

    espera_indicada_segundos NUMERIC,
    ultima_solicitud TIMESTAMPTZ,
    proxima_solicitud_permitida TIMESTAMPTZ,

    CHECK (espera_indicada_segundos >= 0),
    CHECK (codigo_http_robots BETWEEN 100 AND 599)
);

```

-- 2. Ciclos de recopilación e indexación.

```

CREATE TABLE ciclos (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    implementacion TEXT NOT NULL,
    inicio_recopilacion TIMESTAMPTZ NOT NULL
        DEFAULT CURRENT_TIMESTAMP,
    fin_recopilacion TIMESTAMPTZ,
    inicio_indexacion TIMESTAMPTZ,
    fin_indexacion TIMESTAMPTZ,

    estado TEXT NOT NULL DEFAULT 'recopilando',

    CHECK (implementacion IN ('propia', 'librerias')),
    CHECK (
        estado IN (
            'recopilando',
            'indexando',
            'completado',
            'interrumpido',
            'fallido'
        )
    )
);

```

-- 3. URLs administradas por el arañador.

```

CREATE TABLE documentos (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    url_original TEXT NOT NULL,
    url_normalizada TEXT NOT NULL UNIQUE,
    url_final TEXT,
    url_procedencia TEXT,

    fecha_descubrimiento TIMESTAMPTZ NOT NULL
        DEFAULT CURRENT_TIMESTAMP,
    es_semilla BOOLEAN NOT NULL DEFAULT FALSE,

    tipo TEXT,
    estado TEXT NOT NULL DEFAULT 'pendiente',

    intentos INTEGER NOT NULL DEFAULT 0,
    proximo_intento TIMESTAMPTZ,
    ultima_revision TIMESTAMPTZ,
    proxima_revision TIMESTAMPTZ,

    ultimo_codigo_http INTEGER,
    ultimo_error TEXT,
    motivo_descarte TEXT,
    tipo_contenido TEXT,
    idioma_detectado TEXT,
    resultado_extraccion TEXT,

    cantidad_redirecciones INTEGER,
    bytes_ultimo_intento BIGINT,

```

```

id_ciclo BIGINT REFERENCES ciclos(id),

CHECK (
    tipo IN (
        'listado',
        'aviso_analisis',
        'ficha'
    )
),

CHECK (
    estado IN (
        'pendiente',
        'en_proceso',
        'completada',
        'descartada',
        'fallida'
    )
),

CHECK (intentos >= 0),
CHECK (cantidad_redirecciones >= 0),
CHECK (bytes_ultimo_intento >= 0),
CHECK (ultimo_codigo_http BETWEEN 100 AND 599),

CHECK (
    resultado_extraccion IN (
        'correcto',
        'vacio',
        'fallido'
    )
)
);

-- 4. Versiones aceptadas y almacenadas.
CREATE TABLE versiones (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    id_doc BIGINT NOT NULL
        REFERENCES documentos(id) ON DELETE CASCADE,

    fecha_descarga TIMESTAMPTZ NOT NULL,
    titulo TEXT,
    fecha_publicacion TIMESTAMPTZ,
    idioma TEXT NOT NULL,

    ruta_html TEXT NOT NULL,
    ruta_texto TEXT NOT NULL,
    hash_texto VARCHAR(64) NOT NULL,

    tamano_html_bytes BIGINT NOT NULL,
    tamano_texto_bytes BIGINT NOT NULL,

    CHECK (hash_texto ~ '^[0-9a-f]{64}$'),
    CHECK (tamano_html_bytes >= 0),
    CHECK (tamano_texto_bytes >= 0)
);

-- 5. Identificadores y referencia a sus fichas.
CREATE TABLE identificadores (
    codigo TEXT PRIMARY KEY,
    tipo TEXT NOT NULL,
    url_ficha TEXT NOT NULL,

    id_documento_ficha BIGINT
        REFERENCES documentos(id) ON DELETE SET NULL,

    CHECK (tipo IN ('CVE', 'CWE', 'CAPEC')),

    CHECK (
        (tipo = 'CVE' AND codigo ~ '^CVE-[0-9]{4}-[0-9]{4,}$')
        OR
        (tipo = 'CWE' AND codigo ~ '^CWE-[0-9]+$')
    )
);

```

```

        OR
        (tipo = 'CAPEC' AND codigo ~ '^CAPEC-[0-9]+$')
    )
);

-- 6. Menciones: una relación por versión e identificador.
CREATE TABLE menciones (
    id_version BIGINT NOT NULL
        REFERENCES versiones(id) ON DELETE CASCADE,

    codigo_identificador TEXT NOT NULL
        REFERENCES identificadores(codigo),

    PRIMARY KEY (id_version, codigo_identificador)
);

-- Facilita consultar la versión vigente y el historial.
CREATE INDEX idx_versiones_documento_fecha
    ON versiones (id_doc, fecha_descarga DESC, id DESC);

-- Facilita buscar versiones que mencionan un código.
CREATE INDEX idx_menciones_codigo
    ON menciones (codigo_identificador);

-- Facilita localizar tareas pendientes y sus reintentos.
CREATE INDEX idx_documentos_pendientes
    ON documentos (proximo_intento)
    WHERE estado = 'pendiente';

-- Facilita localizar documentos que requieren revisita.
CREATE INDEX idx_documentos_revision
    ON documentos (proxima_revision)
    WHERE proxima_revision IS NOT NULL;

COMMIT;

```

Se usa CHECK en lugar de una tabla por separado, pues, es un proyecto sencillo con estados predeterminados.

Código de Obtención de Vigencia:

```

SELECT *

FROM versiones

WHERE id_doc = 1

ORDER BY fecha_descarga DESC, id DESC

LIMIT 1;

```